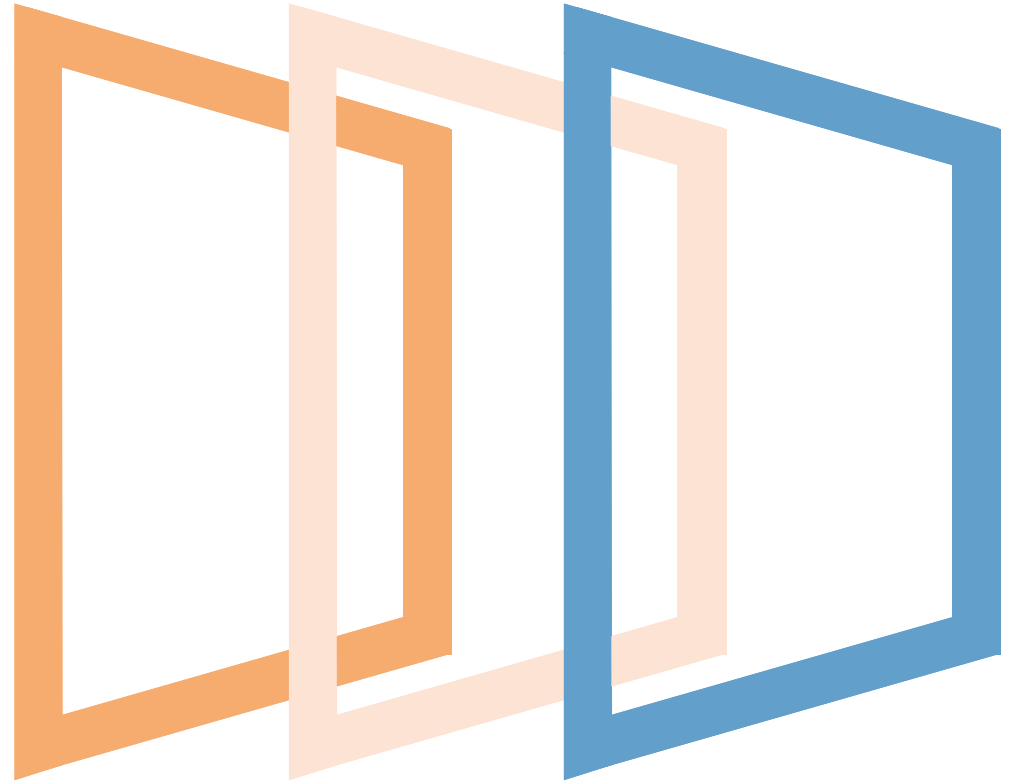


Servlets y JSP

Febrero 2020

minsa1t



An Indra company

Índice

01. HTTP

02. Servlets

03. JSP

04. JDBC Persistencia

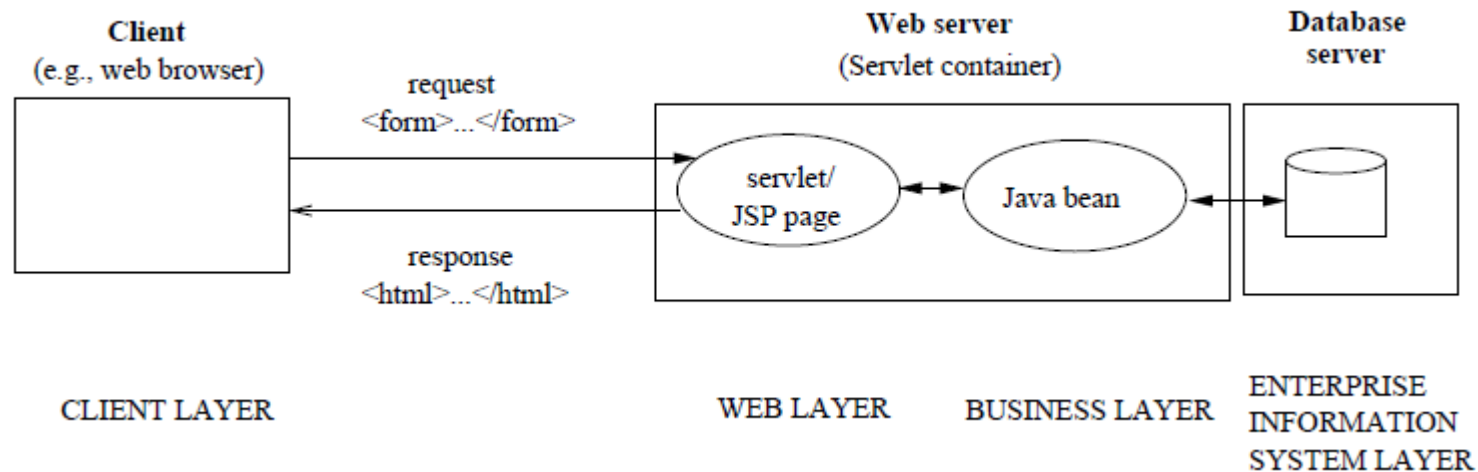
HTTP

01

¿Qué es una aplicación web?

Es una comunicación entre cliente y servidor donde:

- El cliente envía una **petición** HTTP
- El servidor procesa la petición a través de un **servlet** y/o **JSP**, los cuales realizan la lógica de negocio.
- Finalmente devuelve la **respuesta** que se haya procesado.



Petición y respuesta

PETICIÓN

- En la primera línea se incluye el verbo, recurso y versión
- En los encabezados se incluye información sobre el cliente
- El cuerpo que es opcional

Verbo Recurso Versión

↑ ↑ ↑

```
GET /index.html HTTP/1.1
Host: wikipedia.org
Accept: text/html
```

Primera línea

Encabezados

Cuerpo

RESPUESTA

- En la primera línea se incluye la versión y la respuesta
- En los encabezados se incluye información sobre el cliente
- El cuerpo que es opcional

Versión Código respuesta

↑ ↑

```
HTTP/1.1 200 OK
Server: wikipedia.org
Content-Type: text/html
Content-Lenght: 2026

<html>
...
</html>
```

GET y POST

GET

- El concepto GET es obtener información del servidor.
- Los parámetros van en la primera línea. Limitados a lo que cabe en la URL (2048 caracteres).
- Los parámetros sólo pueden ser ASCII.
- Se puede almacenar y cachear.
- Ejemplo de uso: mostrar el detalle de un producto pasándole el identificador.

POST

- POST es enviar información desde el cliente para que sea procesada y actualice o agregue información en el servidor.
- Los parámetros van en el cuerpo.
- Se permite enviar código en binario, lo que incluye ficheros.
- No se puede almacenar ni cachear.
- Ejemplo de uso: Envío de datos para actualizar un producto.

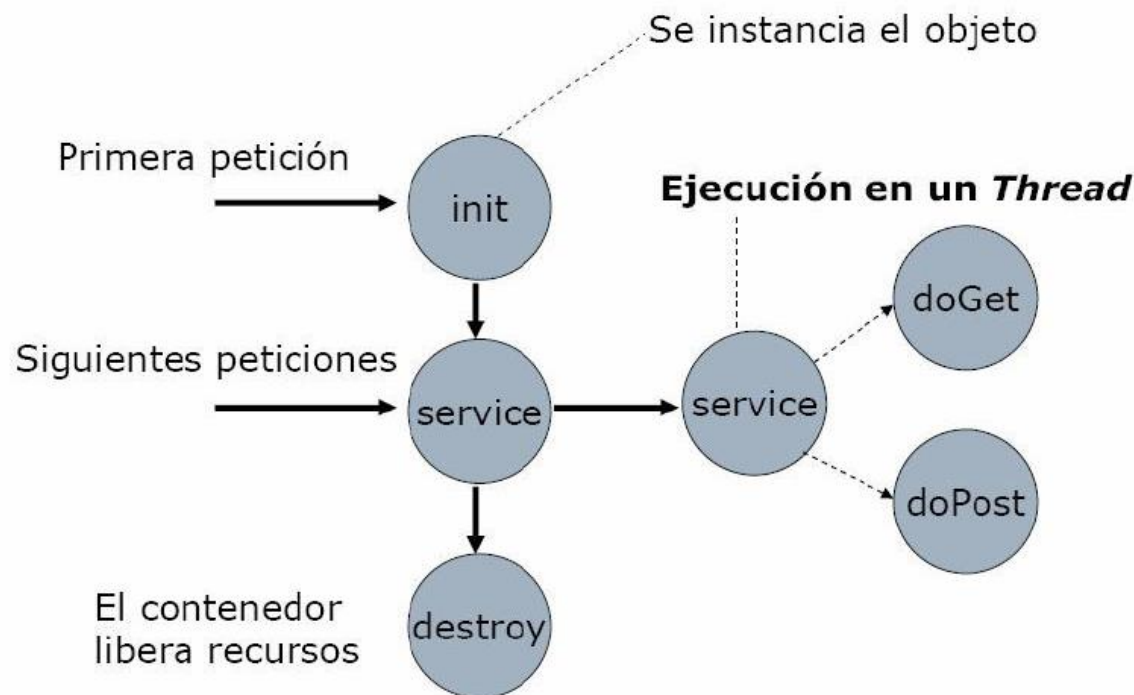
>> Ejecutar Ej1, Ej2 <<

Servlets

02

¿Qué es un servlet?

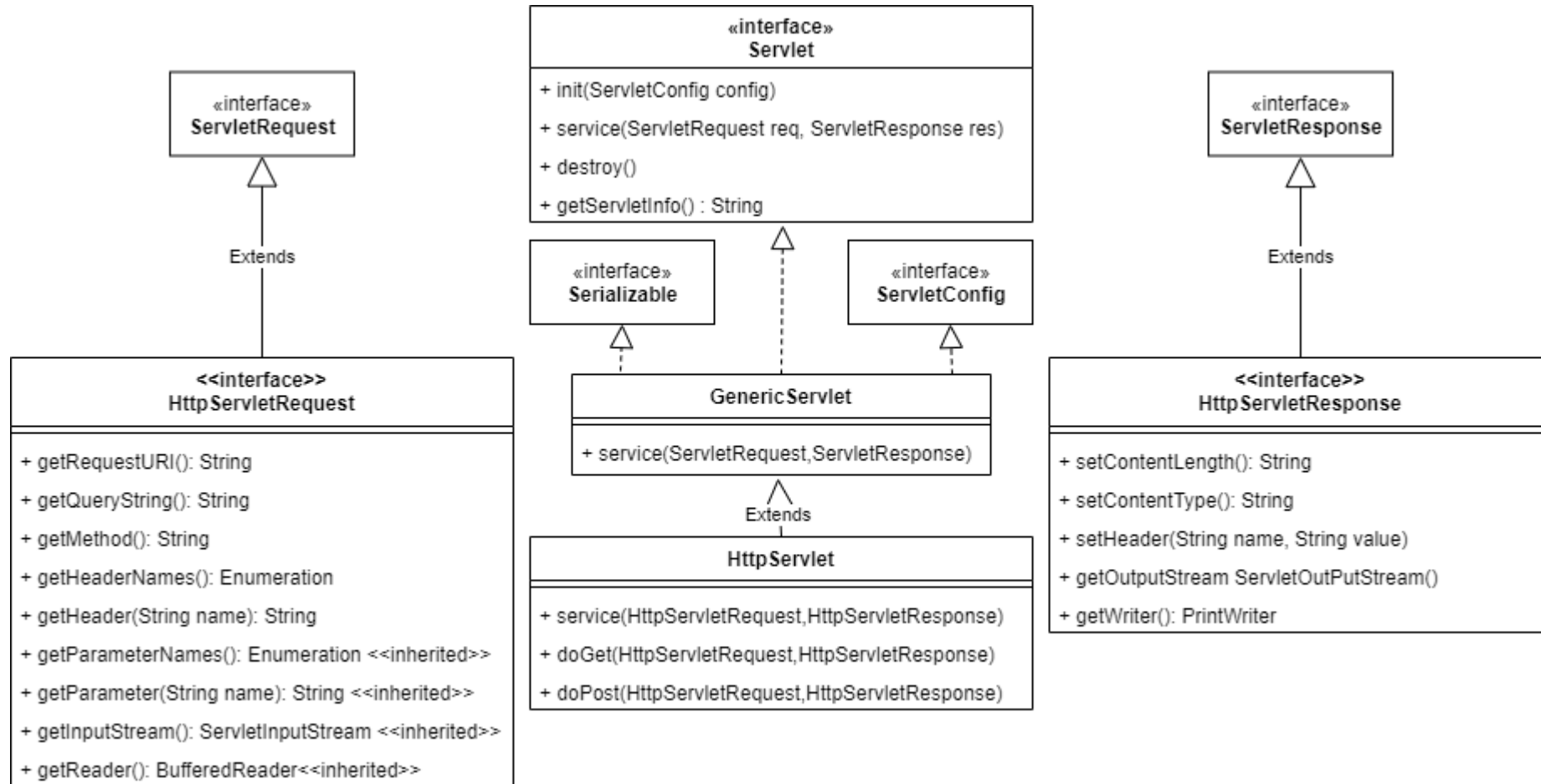
- Un servlet es una clase java que gestiona las peticiones recibidas por el cliente, y les genera una respuesta.
- Los servlets se ejecutan en contenedores de servlets, que traduce las peticiones propias del protocolo a llamadas a objetos java, y traduce las respuestas de los objetos java a respuesta nativa del protocolo; como por ejemplo Tomcat.
- Su ciclo de vida es:



¿Y cómo funciona en JAVA?

Java define una serie de interfaces y clases para trabajar con los servlets, en particular son:

- *Servlet*
- GenericServlet
- HttpServlet
- HttpServletRequest
- HttpServletResponse
- ServletContext



Servlet

Definición

Se trata de una **interfaz** que define los métodos que debe implementar cualquier servlet (independientemente del protocolo).

Métodos

void init(ServletConfig config)

Sólo se llama una vez por el contenedor de servlets, junto antes de resolver la petición.

void service(ServletRequest req, ServletResponse res)

Este método contiene las diferentes operaciones que debe hacer el servlet. Se invoca por cada petición,

void destroy()

Para eliminar el servlet cuando cesen las peticiones del cliente.

java.lang.String getServletInfo()

Devuelve información sobre el servlet: versión, autor...

GenericServlet

Definición

Se trata de una **clase abstracta** que define el protocolo genérico de un servlet. Para usar un servlet HTTP usaremos la clase `HttpServlet` que es una subclase de `GenericServlet`.

Métodos

HttpServlet

Definición

Se trata de una **clase abstracta** con la que tratamos los servlets en el protocolo HTTP. que tiene las operaciones que deben implementarse. Esta será la clase que usaremos como base para crear nuestros propios servlets.

Métodos

void service(HttpServletRequest req, HttpServletResponse res)

Es el responsable de gestionar la petición y generar la respuesta. Aunque no llevan operativa, para eso se desarrollan los métodos doGet y doPost

void doGet(HttpServletRequest req, HttpServletResponse resp)

Se encarga de gestionar una petición del tipo GET

void doPost(HttpServletRequest req, HttpServletResponse resp)

Se encarga de gestionar una petición del tipo POST

HttpServletRequest

Definición

Se trata de una **interfaz** con la que tratamos las peticiones realizadas con el protocolo HTTP y tiene acceso a los elementos que la componen

Métodos

String getRequestURI(): Devuelve el recurso de la primera línea de la petición.

String getQueryString(): Devuelve los parámetros, después de la URL.

String getMethod(): Por ejemplo GET o POST

Enumeration getHeaderNames(): Devuelve las cabeceras

String getHeader(String name): Devuelve la cabecera pasada por parámetro o nulo.

Enumeration getParameterNames(): Devuelve los parámetros

String getParameter(String name): Devuelve el parámetro pasado por parámetro

ServletInputStream getInputStream(): Devuelve el cuerpo en binario

BufferedReader getReader(): Devuelve el cuerpo en texto

Otros: Manejo de atributos, cookies y sesiones

HTTPServletResponse

Definición

Se trata de una **interfaz** que implementa a su vez ServletResponse y encapsula un OutputStream con la respuesta generada.

Métodos

String setContentLength(): Establece la longitud de la respuesta

String setContentType(): Establece el tipo de MIME

setHeader(String name, String value): Setea la cabecera con el nombre y valor.

ServletOutputStream getOutputStream(): Devuelve el Stream para escribir la respuesta con contenido dinámico.

PrintWriter getWriter(): Devuelve el Stream para escribir la respuesta con texto plano.

>> Ejecutar Ej3, Ej4, Ej5 <<

Dificultades con los servlets

- Un servlet es capaz de generar una página web, pero el resultado no es intuitivo, es difícil de generar y de mantener.
- No existe una separación entre modelo y vista.
- Cada servlet es una clase completa con diferentes métodos para gestionar el ciclo de vida de los servlets.

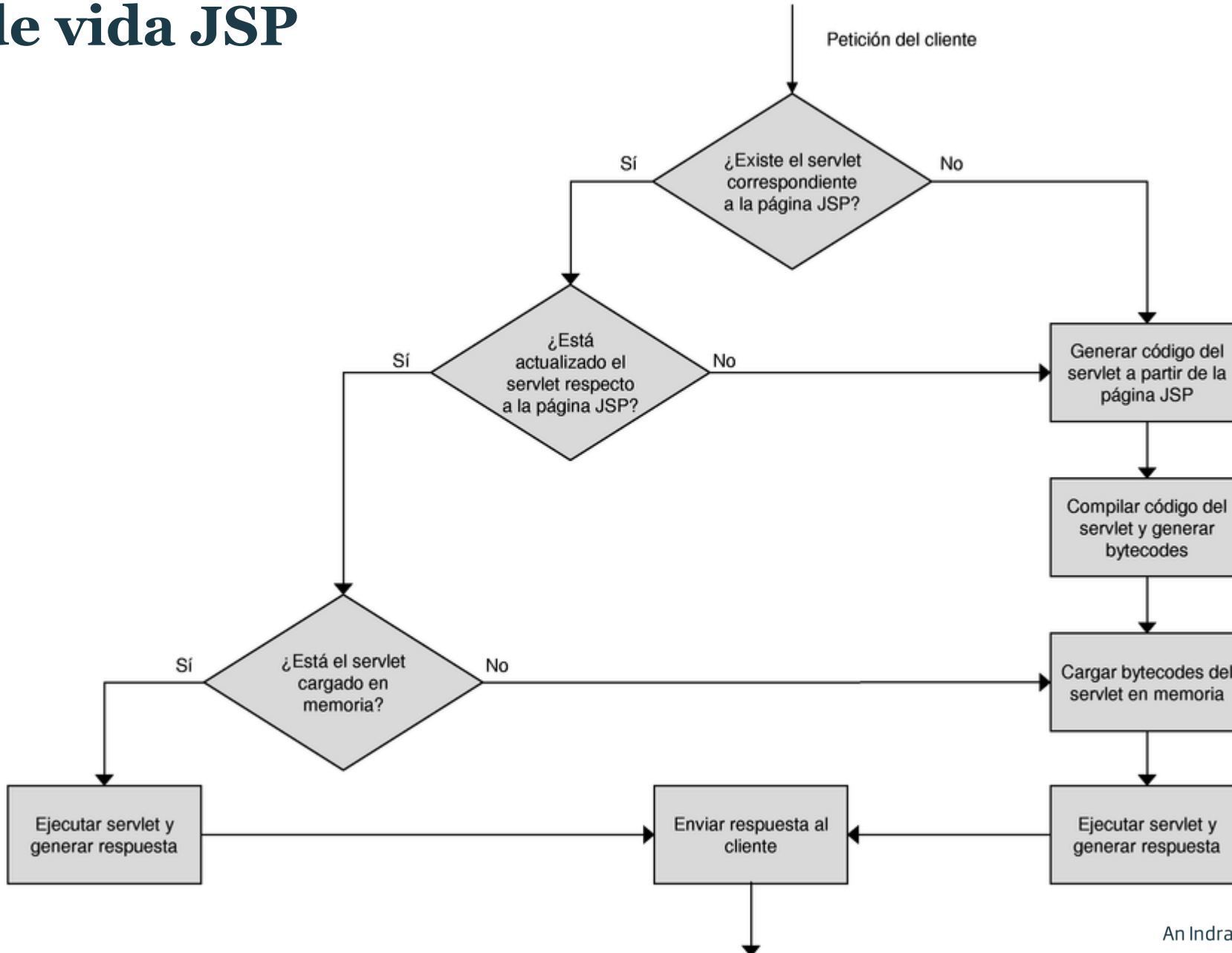
JSP

03

¿Qué es una JSP?

- Es una página web, normalmente escrita en HTML que contiene contenido dinámico (expresiones en java, tags y java beans)
- Una JSP que traduce automáticamente en un servlet. Cuando se solicita la JSP, el servlet asociado se ejecuta automáticamente
- El resultado de la ejecución es:
 - Código estático que ya existía en la HTML
 - Resultado de la ejecución de las partes escritas en JAVA.
- Las JSP se ejecutan dentro de un contenedor de JSP, por ejemplo TOMCAT

Ciclo de vida JSP



Componentes de una JSP (I)

- Directiva: Palabras reservadas para especificar atributos de nuestra página
 - Page: para definir las propiedades de la página JSP que estemos realizando. Esta directiva la podemos usar varias veces a lo largo de la pagina JSP, pero solo podemos asignar el valor de un atributo una vez por cada uno excepto para el atributo import
 - Include: Se usa para incluir un fichero dentro de la propia pagina JSP
 - TagLibs: Declara la librería de etiquetas a usar. Lo veremos más adelante.
- Expresiones: contienen variables en java a las que se puede acceder (una única línea):

Current time is <%= new java.util.Date() %>

- Scriptlet: Bloque que contiene varias instrucciones en java.

>> Ejecutar Ej6 <<

Componentes de una JSP (II)

- **Declaraciones:** Permiten configurar variables para su uso posterior en expresiones o scriptlets. También podemos declarar variables dentro de expresiones o scriptlets en el momento de usarlas.
 - Sintaxis: `<%! code %>`
- **Variables predefinidas:** Hay una serie de variables propias de las paginas JSP a las que se puede acceder directamente. Por ejemplo:
 - `request`: La instancia de la clase `HttpServletRequest` que contiene la petición.
 - `response`: La instancia de la clase `HttpServletResponse` que contiene la petición.
 - `out`: Es el `PrintWriter` que nos sirve para escribir en la respuesta que el daremos al cliente.

>> Ejecutar Ej1, Ej7, Ej2, Ej8, Ej9 <<

Diferencias entre Servlet y JSP

Servlet

- Es JAVA puro.
- Si tiene que devolver una respuesta, genera el HTML.
- Soporta una lógica más compleja.

JSP

- Es HTML mezclado con JAVA
- Se utiliza cuando la lógica a programar es sencilla.

>> Ejercicio piedra, papel, tijeras, lagarto, Spock <<



Piedra, Papel, Tijeras, Lagarto, Spock

Tijeras cortan papel
Papel tapa a piedra
Piedra aplasta a lagarto
Lagarto envenena a Spock
Spock rompe tijeras
Tijeras decapitan lagarto
Lagarto devora papel
Papel desautoriza Spock
Spock vaporiza piedra
Piedra aplasta a tijeras

Problemas con las JSP

- Hay código JAVA mezclado, es complicado de leer y no es elegante → Vamos a usar tags y java beans
- La presentación está mezclada con el negocio → Tags, java beans y patrones de diseño.
- Falta de abstracción. Por ejemplo las conexiones de BBDD irían directamente en la JSP → Patrones de diseño.

Librerías de tags

- Las etiquetas de JSP pueden ser definidas para que tengan un comportamiento específico.
 - `<util:getValue attribute="name" />`
- Existen ya librerías disponibles para ser usadas en JSP.
 - `<jsp:useBean>`
 - `<jsp:setProperty>`
 - `<jsp:getProperty>`
 - `<jsp:include>`
 - `<jsp:forward>`
 - `<jsp:param>`

```
<jsp:forward page="aux.jsp">  
  <jsp:param name="customer" value="Pepet"/>  
</jsp:forward>">
```

```
<jsp:include page="aux.jsp" flush="true">  
  <jsp:param name="customer" value="Pepet"/>  
</jsp:forward>">
```


JSTL - JavaServer Pages Standard Tag Library

- Está constituida por 4 librerías:
 - c: Control de flujo, imports...
 - x: Xml para su procesamiento.
 - fmt: l18n para la internacionalización y formatos de fechas, importes
 - sql: Para gestionar el acceso a base de datos
- Un lenguaje de expresión que nos permite evaluar pequeñas expresiones.
 - `${1+2+4}`
 - `${a==222}`
 - `${a<=222}`
 - `${param.nom}`

>> Descargar librerías <<

>> Ejecutar Ej10, Ej11 <<

JAVA BEANS

- Son clases que cumplen con las siguientes características.
 - Tienen un constructor vacío.
 - Implementan la interfaz Serializable o Externalizable
 - Tienen atributos y métodos.
- Su objetivo es facilitar el uso de clases en las aplicaciones web.

>> Ejecutar Ej12 <<

JDBC Persistencia

04

JDBC (I)

- JDBC es un package que gestiona las conexiones con la base de datos.
- Los diferentes tipos de BBDD generan una librería llamada Driver, que los desarrolladores usamos para gestionar las conexiones sobre la base de datos.
- Su funcionamiento es:

- Cargar el driver.

```
Class.forName("specificJDBCdriver");
```

- Abrimos una conexión

```
Connection con=  
    DriverManager.getConnection("jdbc:DatabaseURL");
```

JDBC (II)

- Crear la SQL con Statement

```
Statement stm=con.createStatement();  
rs=stm.executeQuery("Select xxx from table.....");
```

- Recuperar el resultado (ResultSet) y gestionarlo como queramos:

```
while (rs.next()){  
    name=rs.getString("name");  
    amount=rs.getInt("amount");  
}
```

>> Descargar driver Oracle <<

>> Ejecutar Bookstore <<

Práctica final (2ª Parte)

- Añadir persistencia para que el catálogo quede en BBDD
- Mostrar el listado del catálogo a través de una jsp.
- Acceder al detalle de cada una de las películas a través de un link.
- Insertar nuevas películas.

¡Gracias!

Presentación:

Vanessa Andres Blas

vablas@minsait.com

Parc tecnològic de Gardeny, H3, 1^a

25003 Lleida

Lleida España

T +34 973 19 35 16

E 240316

www.minsait.com

minsait

An Indra company

minsait

Mark Making the way forward

An Indra company